



# **MAVEN AUTOMATION TOOL**

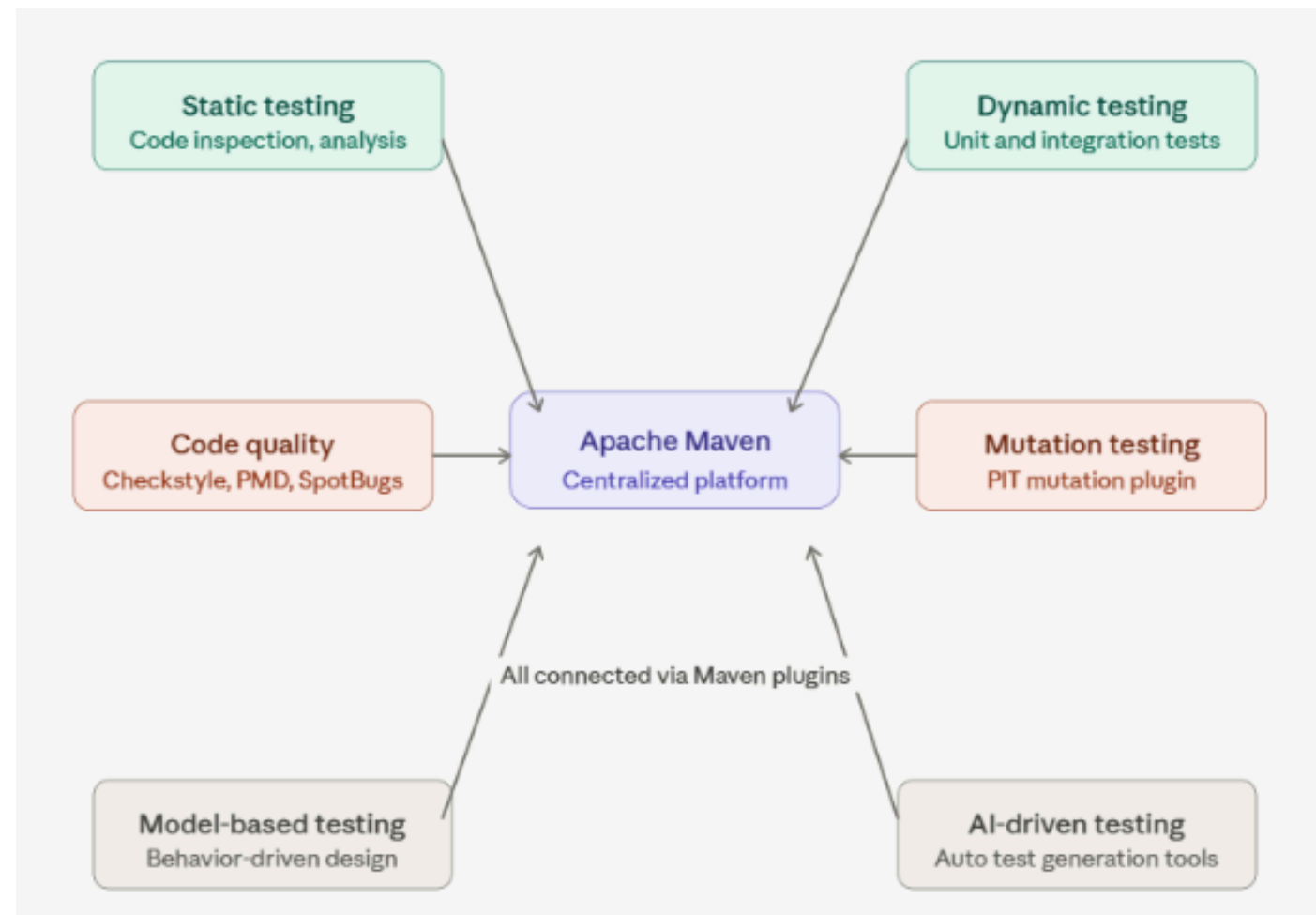
Maven is an automation and management tool developed by Apache Software Foundation. ■

# INTRODUCTION

- **Before Maven:** Developers wrote custom scripts for compiling, testing, downloading libraries, and packaging.
- **With Maven:** One standardized command (`mvn package`) automates all steps in the right order.
- Saves time, reduces errors, ensures consistency across teams.
- Beyond just building, Maven **manages** your entire **project's ecosystem**: which external libraries you need, what versions of those libraries to use, how your project relates to other projects, and what metadata describes your project (name, version, author).
- **Apache Software Foundation**- Non-Profit organization that maintains hundreds of open source projects



# CENTRALIZED PLATFORM



Apache Maven serves as a **centralized build and project management platform** that automates and **integrates** multiple software quality activities, including static testing (code inspection and static analysis), dynamic testing (unit and integration testing), code quality analysis, mutation testing, model-based testing, and AI-driven testing, through the use of specialized **plugins, frameworks, and external testing tools within a unified development workflow**

| Language  | Maven Plugin       |
|-----------|--------------------|
| C# / .NET | NPanday plugin     |
| Scala     | scala-maven-plugin |
| Ruby      | jruby-maven-plugin |
| Groovy    | gmaven-plus        |
| C/C++     | maven-nar-plugin   |
| PHP       | php-maven          |

It is written in Java Language and used to build and manage projects written in C#, Ruby, Scala, and other languages.

Maven tool's own source code is written in Java, and you need a Java Runtime Environment (JRE) installed on your machine to run Maven. However, Maven is not limited to building Java projects. Maven is a general build framework. Through its plugin system, it can compile, test, and package projects written in many other languages:



It helps to **build projects, manage dependency, and documentation.**



**Build projects** - Maven does this through its lifecycle phases- validate, compile, test, package, verify, install, deploy, running each step in order automatically.

**mvn package** -This one command does ALL of these automatically:

- # 1. Validates your project config
- # 2. Downloads any needed libraries
- # 3. Compiles your .java files into .class files
- # 4. Runs all your tests
- # 5. Packages everything into a .jar file



It helps to **build projects, manage dependency, and documentation.**

**Dependency management** - In any real project you'll use dozens of external libraries. Without Maven, you'd manually download each .jar file, put it in the right folder, and track versions yourself. If Library A needs Library B version 2.1, but Library C needs Library B version 2.3, you'd have a conflict to resolve by hand.

```
my-app:jar:1.0
+- jackson-databind:jar:2.15.2
| +- jackson-core:jar:2.15.2      ← Auto-fetched, you didn't ask for this
| \- jackson-annotations:jar:2.15.2 ← Auto-fetched, you didn't ask for this
```



It helps to **build projects, manage dependency, and documentation.**



**Documentation** - Maven can generate a complete project website with documentation automatically using **mvn site**. This includes:

- Javadoc (API documentation from your code comments)
- Test coverage reports
- Dependency reports (which libraries you use and their licenses)
- Code quality reports
- Project information (name, version, team, etc.)



# PROCESSES WHICH CAN MANAGE USING MAVEN

## 1. BUILDS

The build process is Maven's most fundamental job. "Building" means taking your raw human-written source code and converting it into something a computer can actually run or deploy.

The build is divided into a strict sequence of phases. You cannot skip earlier ones – if you ask for package, Maven automatically runs validate → compile → test → package in order.

## 2. DOCUMENTATION

Documentation in Maven means automatically generating human-readable reference material directly from your source code and project configuration, without you having to write it manually in a separate tool.

The main mechanism is Javadoc. When you write special `/** ... */` comments in your Java code, Maven can harvest those comments and turn them into a navigable HTML website. This is called the "project site."





# PROCESSES WHICH CAN MANAGE USING MAVEN

## 3. REPORTING

Reporting is closely related to documentation but focuses specifically on code quality metrics and test results.

The difference in plain terms: documentation explains what your code does; reports measure how well it does it.

## 4. DEPENDENCIES

Dependencies are the external libraries your project needs to function.

Before Maven, adding a library meant: download the .jar file manually, figure out what other .jar files that library itself needs, download those too, put them all in the right folder, and add them to your build script. One library might need five others, each of which needs three more. This was called "JAR hell."

Maven solves this completely. You simply declare what you need, and Maven handles everything:



# PROCESSES WHICH CAN MANAGE USING MAVEN

## 5. SCMs (Source Code Management)

### Maven + Version Control (Git, SVN, Mercurial)

Maven integrates with these systems so that your `pom.xml` knows exactly where your project's source code lives.

This integration matters because Maven needs to know your SCM location to perform releases (tagging versions in Git), generate source links in reports (so developers can click a class name and see its source in a web browser), and enable CI/CD pipelines to check out the right code.

## 6. RELEASES

A "release" in software means publishing an official, stable, versioned snapshot of your project. Before Maven, this involved many manual steps:(update version, build, copy jar, tag in Git, set next version)

With Maven: just two commands.

- `mvn release:prepare` → updates version, commits, tags in Git, sets next snapshot.
- `mvn release:perform` → builds & deploys release.



# **POPULAR IDE'S SUPPORTING DEVELOPMENT WITH MAVEN**

- Eclipse
- IntelliJ IDEA
- Jbuilder
- NetBeans
- MyEclipse

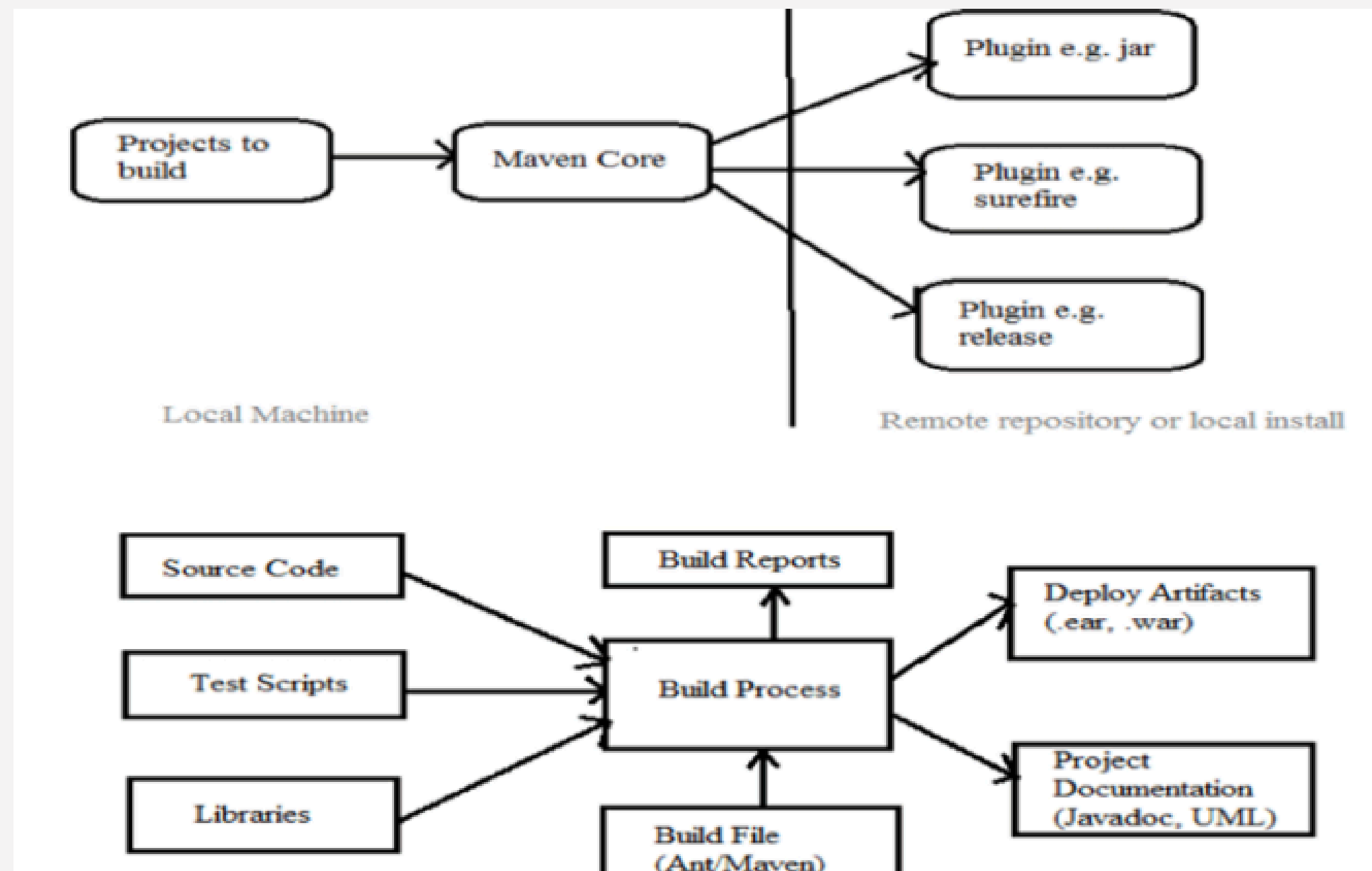


# **MAVEN ARCHITECTURE OVERVIEW**

When Maven receives a project to build, it doesn't do everything itself. It delegates specific jobs to plugins (small programs that know how to do one task well). These plugins live in a remote repository (like Maven Central on the internet) or a local install on your machine.

- **JAR plugin** → knows how to package code into a .jar file
- **Surefire plugin** → knows how to run JUnit test cases
- **Release plugin** → knows how to prepare a release version of your app

**Maven Core acts like a coordinator:** it reads your project requirements from pom.xml, figures out which plugins are needed, fetches them from the remote/local repository, and then uses them.



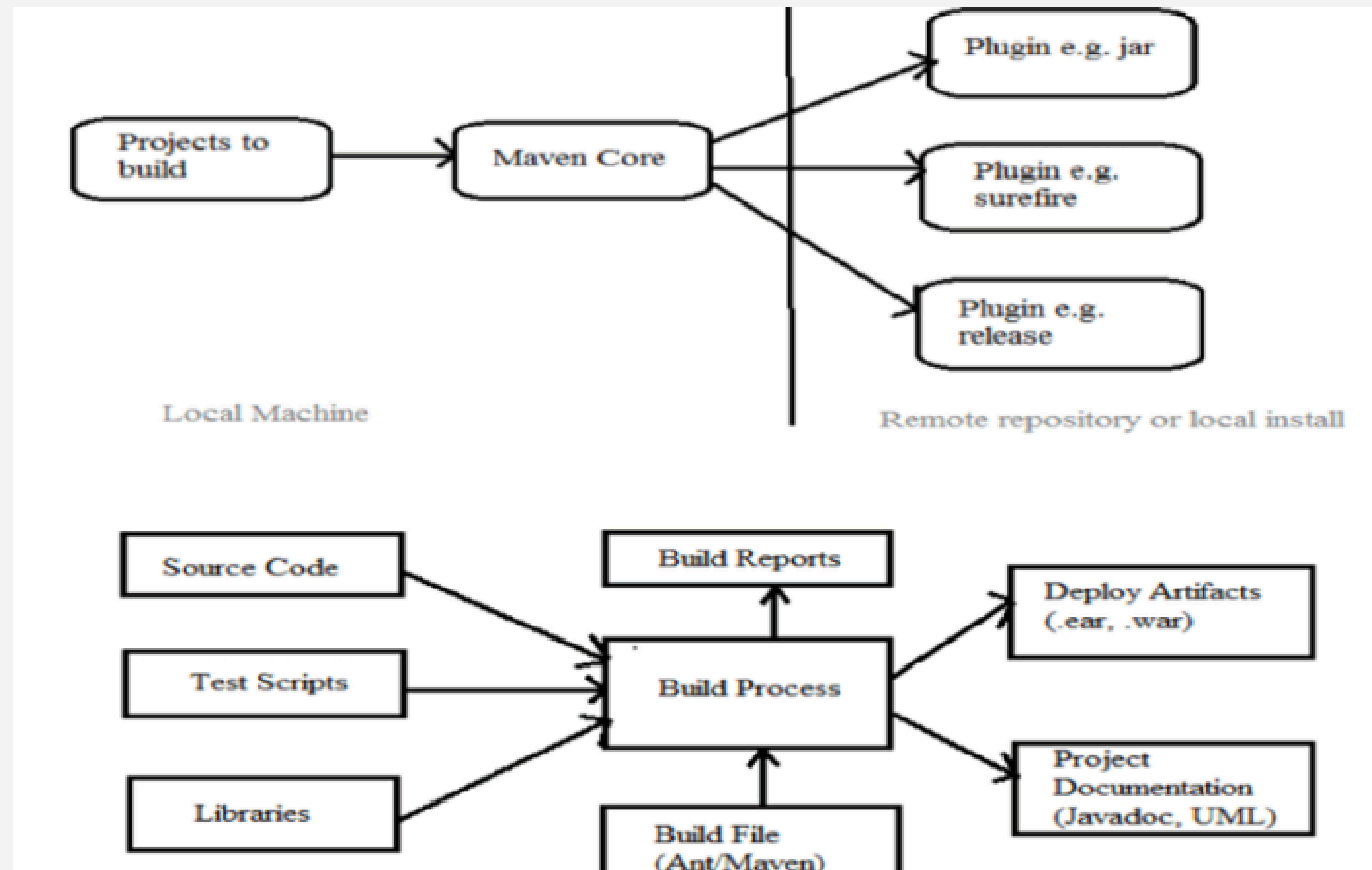


Your project is made of three types of inputs:

- **Source Code**- your Java .java files
- **Test Scripts**- your unit/integration tests
- **Libraries**- third-party dependencies (defined in pom.xml)

These three inputs feed into the Build Process, which is orchestrated by a Build File (Maven's pom.xml). The build process produces two kinds of output:

- **Deploy Artifacts (.ear, .war, .jar)** – the runnable application
- **Project Documentation** (Javadoc, UML diagrams) – auto-generated documentation
- **Build Reports** – test results, code coverage, etc.

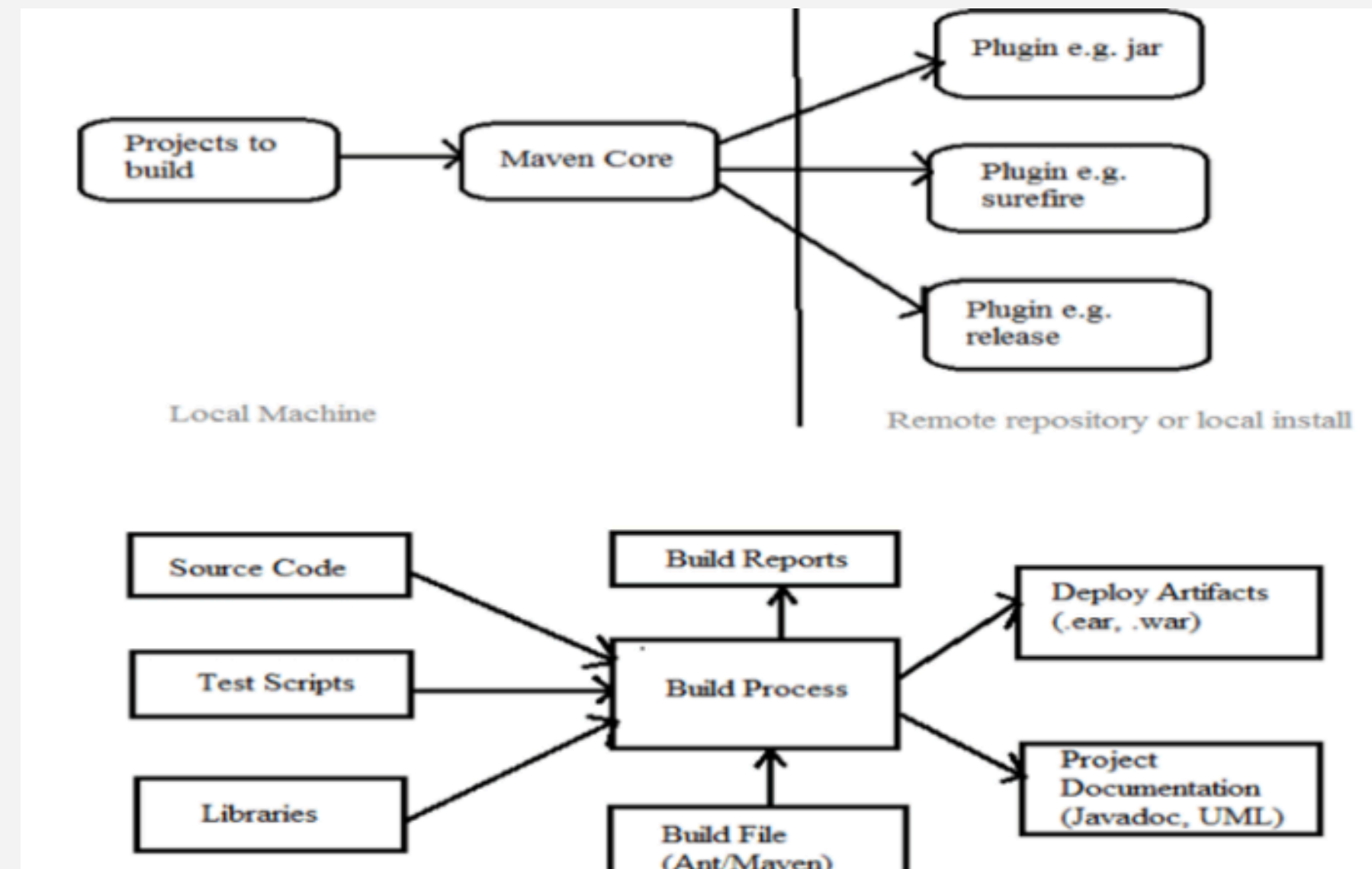




# **MAVEN ARCHITECTURE STEPS**

**Step 1 – Project to Build:** You hand Maven your project. The key file here is pom.xml (Project Object Model) – this XML file is the "blueprint" that tells Maven everything about your project: its name, version, what libraries it needs, what plugins to use, etc.

**Step 2 – Maven Core reads pom.xml:** Maven Core is the brain of Maven. It reads the project details and decides: what to compile, what to test, what libraries are needed, what package to create (Maven Core reads the pom.xml file and understands project requirements)



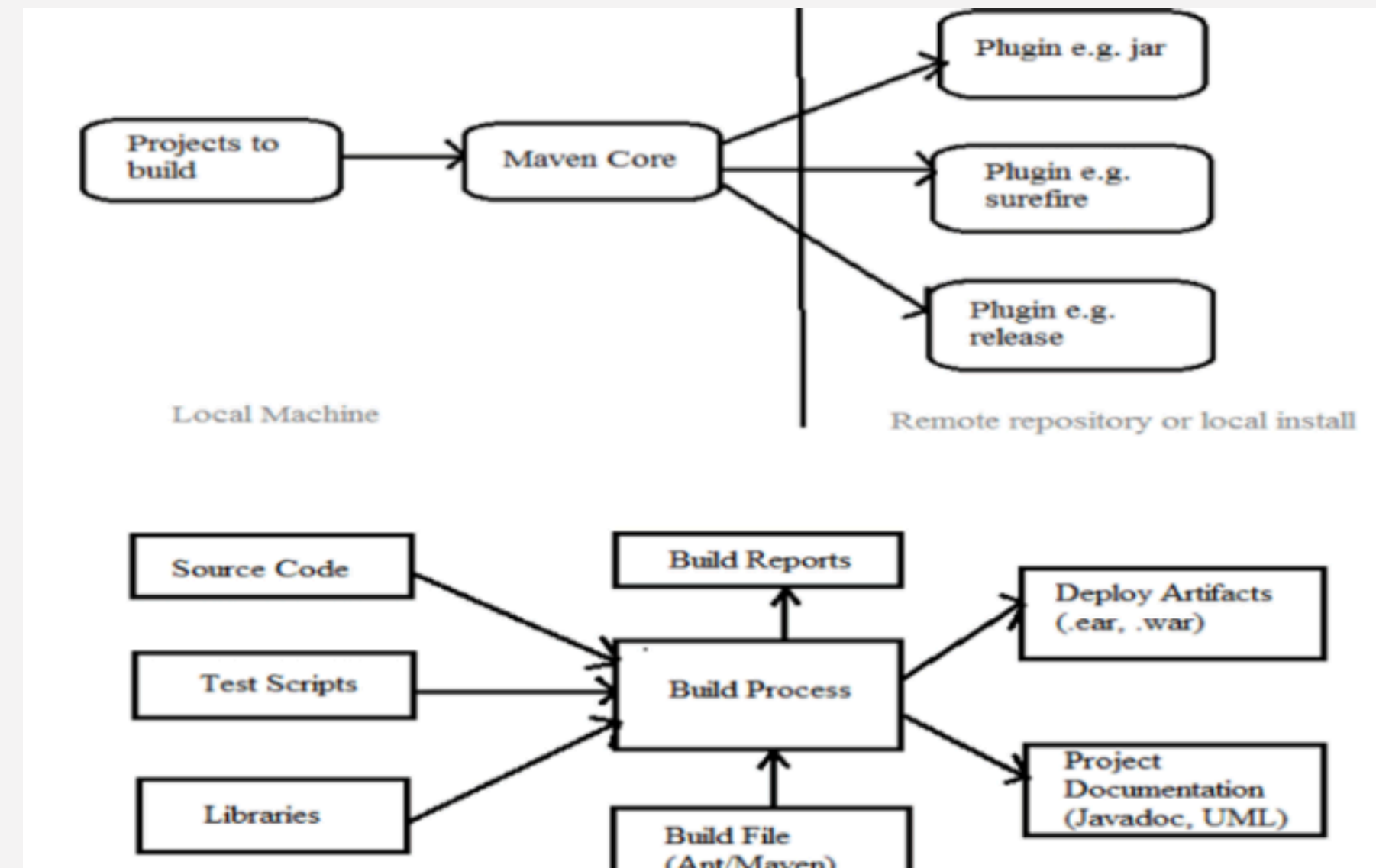


**Step 3 – Dependency Resolution:** If libraries are missing, Maven downloads them from local or remote repositories. If your pom.xml says "I need Spring Boot version 3.2", Maven checks your local cache (~/.m2/repository) first. If it's not there, Maven downloads it from Maven Central (the public remote repository). This is why Maven projects work on any machine – dependencies are declared, not bundled.

**Step 4 – Plugins:** Each phase of the build is handled by a plugin. You don't write these plugins – Maven comes with sensible defaults (called the "default lifecycle"). You only configure them in pom.xml if you need custom behaviour.

Examples:

- JAR plugin → creates .jar file
- Surefire plugin → runs test cases
- Release plugin → prepares final release

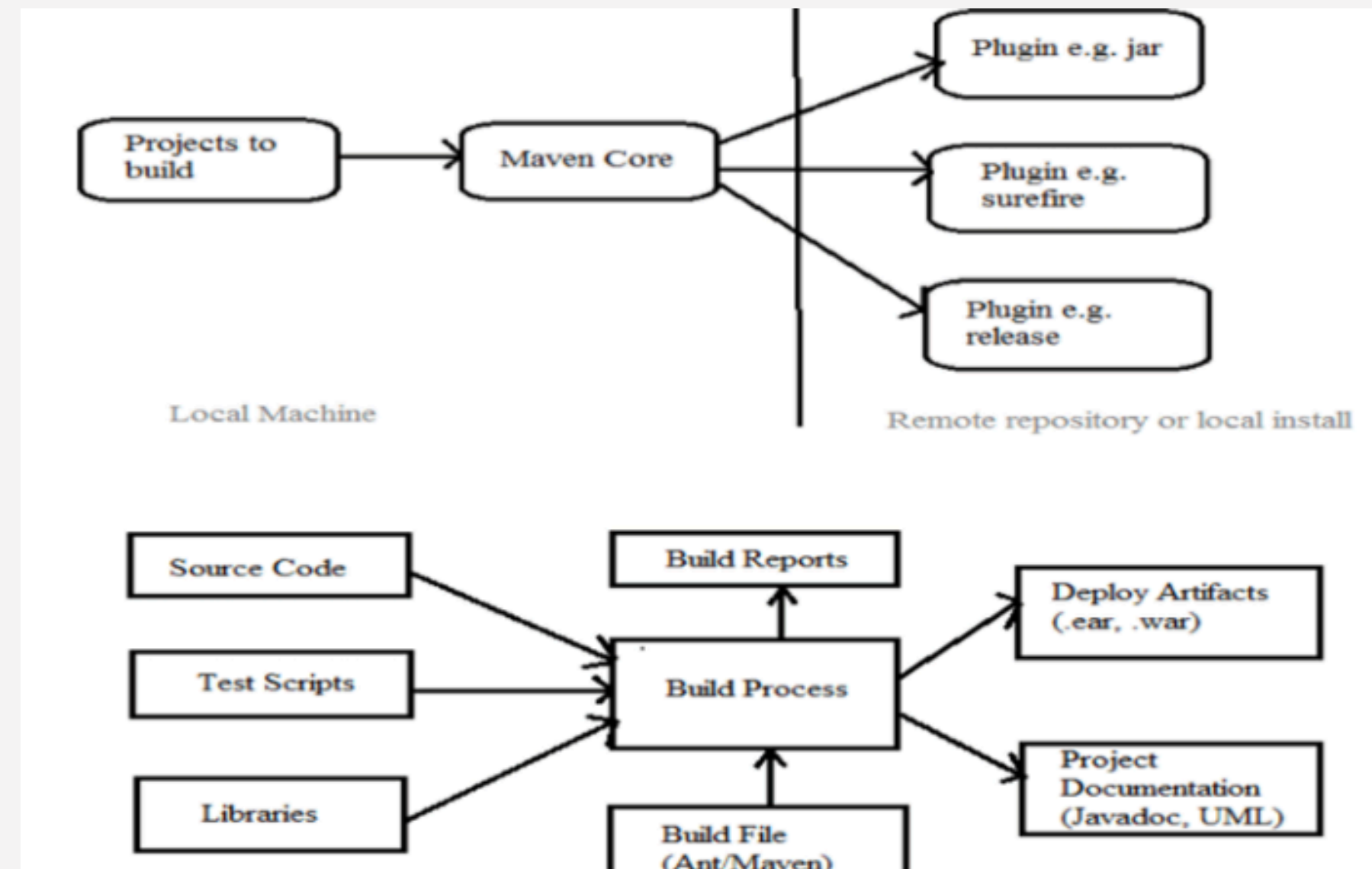


**Step 5 – Build Execution:** Maven runs the build lifecycle in order:

1. Compile source code (javac)
2. Execute test cases (Surefire plugin runs JUnit)
3. Package the compiled code (JAR/WAR/EAR plugin)

If any test fails, Maven stops and reports the failure – it won't package broken code.

**Step 6 – Output:** A successful build produces your deployable artifact (e.g. myapp-1.0.war) plus documentation and reports.





# SUMMARY

- 
- **"Why do we need pom.xml?"** – It's the single source of truth. Without it, Maven doesn't know what to build or how.
  - **"Where do plugins come from?"** – They're downloaded from Maven Central (an online repository) the first time and cached locally in `~/.m2`.
  - **"What's the difference between .jar, .war, and .ear?"** – JAR is a plain Java library/app, WAR is a web application (goes on a web server like Tomcat), EAR is an enterprise app bundle containing multiple WARs/JARs.
  - **"What if a test fails?"** – Maven halts the build at the test phase and reports the failure. Nothing gets packaged.